

Problem Statement:

A university maintains an automated log of attendance activities recorded in order.

Each record represents a student being **Present (P)** or **Absent (A)**.

A **stable attendance window** is defined as a continuous period in which:

- The number of **Present** and **Absent** records are equal.
-

Input Format

1. Integer **N** — number of attendance records
 2. **N** entries representing attendance status
 - **P** → Present
 - **A** → Absent
-

Output Format

- Print a single integer: the **maximum length of a stable attendance window**
 - If no such window exists, print **0**
-

Algorithm: Stable Attendance Window

Step 1:

Start

Step 2:

Read integer **N** (number of attendance records)

Step 3:

Read **N** attendance entries (**P** or **A**)

Step 4:

Initialize:

- `sum = 0`
- `maxLength = 0`

- Create an unordered_map<int, int> mp

Step 5:

Insert initial value:

- mp[0] = -1
(to handle subarray starting from index 0)

Step 6:

For each index i from 0 to N-1:

- If attendance is P, update sum = sum + 1
- If attendance is A, update sum = sum - 1

Step 7:

Check if sum exists in mp:

- If **yes**:
 - length = i - mp[sum]
 - Update maxLength = max(maxLength, length)
- If **no**:
 - Store mp[sum] = i

Step 8:

After loop completion, print maxLength

Step 9:

Stop

Time Complexity

- **O(N)**
-

Code:

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int main() {
```

```
    int N;
```

```
cin >> N;
vector<char> attendance(N);
for (int i = 0; i < N; i++) {
    cin >> attendance[i];
}
unordered_map<int, int> mp;
int sum = 0;
int maxLen = 0;
mp[0] = -1;
for (int i = 0; i < N; i++) {
    if (attendance[i] == 'P')
        sum += 1;
    else if (attendance[i] == 'A')
        sum -= 1;
    if (mp.find(sum) != mp.end()) {
        int length = i - mp[sum];
        maxLen = max(maxLen, length);
    }
    else {
        mp[sum] = i;
    }
}
cout << maxLen << endl;
return 0;
}
```